

Relatório Técnico: Sistemas Embarcados

Identificação do Aluno	Número USP	Data de Entrega
Giovanni Antonio Silva de Oliveira	14575064	xx/xx/202x

1. Identificação do Experimento

- **Título:** Atividade 7 - Shared Resources
- **Disciplina:** PSI3441 - Arquitetura de Sistemas Embarcados
- **Professor:** Gustavo Pamplona Rehder

2. Respostas, Comentários e Análises

1. Parte 1 - Sem Sincronização

Nessa parte da atividade, é observável que ocorrem alguns conflitos e imprevistos na execução do código. Para numerar elas:

- Ao iniciar a placa, se a Thread tem valores iguais ou a do cliente tem prioridade maior, o primeiro print da tela tem como saldo um valor negativo, em decorrência de que ela foi a primeira thread a rodar, antes da do padeiro adicionar.
- Como nesse caso elas foram feitas de forma a serem preemptivas, caso deixemos as prioridades dentre elas distintas, primeiramente a impressão no terminal serial é corrompida, pois durante a transmissão uma task tomou posse do serial no meio da utilização, e segundo que por vezes o valor do terminal não era atualizado direito, fazendo com que em certos momentos sejam utilizados valores antigos ou então agregados novos a variado com base em antigos que foram atualizados na última iteração

2. Parte 2 - Somente Mutex

- Nessa segunda parte, os problemas da primeira são parcialmente resolvidos, já que a variável, e consequentemente o terminal, não são

corrompidos, já que há a trava do mutex que, caso alguma outra task esteja usando o recurso, aguarda liberá-lo antes de usar. não deixando uma corrida e nem concorrente o uso do recurso.

- Contudo, caso a prioridade do cliente seja maior que a do padeiro, no início continua ocorrendo o saldo negativo de pães, que ainda está errado.

3. Parte 3 - Mutex + Semáforo

Por fim, nessa última parte é utilizado os recursos tanto do semáforo quanto do mutex para resolver todos os problemas da primeira parte da atividade, no qual:

- O problema que ainda ocorria na parte anterior foi resolvido com sucesso, onde agora o número de pães não fica mais negativo, mesmo mudando a prioridade, e também há um limite de produtos na vitrine agora, e se esse contador do semáforo atinge algum desses limites, a task é colocada em repouso naquele ponto até a atividade sair daquele limite. assim resolvemos todos os problemas que a parte 1 tinha.

4. Comparação

- a. A proteção dos recursos ocorre com o Mutex nesse caso, no qual ele só deixa uma task acessar a disponibilidade por ao mesmo vez, impedindo corrupção do dado, e a disponibilidade é pelo semáforo, que impõem limites para os valores da variável, e caso o limite seja atingido, impede que uma task ocorra naquele momento.
- b. O Mutex já era suficiente na parte 2, **desde que a prioridade do cliente fosse menor a do padeiro, e o padeiro sempre tenha um tempo maior de produção que os de cliente (ou por tempo suficiente para que o resultado não ultrapasse 0).**
- c. Como solução completa e robusta do problema, o semáforo se apresentou mais adequado na gestão das atividades.
- d. Com Mutex e Semáforo pela robustez observada de sem número negativo e limite físico da prateleira.

3. Código-Fonte

Parte 1 - Sem Sincronização

```
////////////////////////////////////
////////////////////////////////////Main.c
////////////////////////////////////

#include <zephyr/kernel.h>

// Tamanho de memória para cada thread
#define STACK_SIZE 1024

// prioridade da thread
#define PADEIRO_THREAD_PRIORITY 7
#define CLIENTE_THREAD_PRIORITY 7

// para alternar os modos do botao
static volatile int saldo_vitrine = 0;

//////////////////////////////////// thread
padeiro////////////////////////////////////
void thread_padeiro_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        saldo_vitrine++;

        printk("[PADEIRO] Produziu 1 pao. Saldo atual da vitrine:
%d\n", saldo_vitrine);

        k_msleep(1000); // Aguarda 1 segundo
    }
}

//////////////////////////////////// thread
cliente////////////////////////////////////
void thread_cliente_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        saldo_vitrine--;
```

```

        printk("[CLIENTE] Retirou 1 pao. Saldo atual da vitrine: %d\n",
saldo_vitrine);

        k_msleep(1500); // Aguarda 1.5 segundos
    }
}

//K_THREAD_DEFINE(
//    nome_da_thread,        // Um nome interno para a thread
//    tamanho_da_pilha,      // Memória reservada (geralmente 1024 bytes)
//    funcao_da_thread,      // O nome da função que tem o loop da thread
//    arg1, arg2, arg3,      // Argumentos (geralmente NULL)
//    prioridade,            // Número da prioridade (menor = mais
importante)
//    opcoes,                // 0 por padrão
//    delay_inicial          // 0 para iniciar na hora
//);

// declaratacao thread adc
K_THREAD_DEFINE(
    padeiro_tid,
    STACK_SIZE,
    thread_padeiro_fn,
    NULL,
    NULL,
    NULL,
    PADEIRO_THREAD_PRIORITY,
    0,
    0
);

// declaratacao thread accel
K_THREAD_DEFINE(
    cliente_tid,
    STACK_SIZE,
    thread_cliente_fn,
    NULL,
    NULL,
    NULL,
    CLIENTE_THREAD_PRIORITY,
    0,
    0
);

```

```

int main(void)
{
    printk("=====\n");
    printk(" PSI3441 - Atividade 7 - Parte 1 (Sem Sincronizacao)\n");
    printk("=====\n\n");

    // O main entra em repouso e deixa as threads rodando
    while (1) {
        k_sleep(K_FOREVER);
    }
    return 0;
}

```

Parte 2 - Somente Mutex

```

////////////////////////////////////
////////////////Main.c
////////////////////////////////////

#include <zephyr/kernel.h>

// Tamanho de memória para cada thread
#define STACK_SIZE 1024

// prioridade da thread
#define PADEIRO_THREAD_PRIORITY 7
#define CLIENTE_THREAD_PRIORITY 5

// para alternar os modos do botao
static volatile int saldo_vitrine = 0;

K_MUTEX_DEFINE(vitrine_mutex);

//////////////////////////////////// thread
padeiro////////////////////////////////////
void thread_padeiro_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        k_mutex_lock(&vitrine_mutex, K_FOREVER);

        saldo_vitrine++;
    }
}

```

```

        printk("[PADEIRO] Produziu 1 pao. Saldo atual da vitrine:
%d\n", saldo_vitrine);

        k_mutex_unlock(&vitrine_mutex);

        k_msleep(1000); // Aguarda 1 segundo
    }
}

//////////////////////////////// thread
cliente////////////////////////////////
void thread_cliente_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        k_mutex_lock(&vitrine_mutex, K_FOREVER);

        saldo_vitrine--;

        printk("[CLIENTE] Retirou 1 pao. Saldo atual da vitrine: %d\n",
saldo_vitrine);

        k_mutex_unlock(&vitrine_mutex);

        k_msleep(1500); // Aguarda 1.5 segundos
    }
}

//K_THREAD_DEFINE(
//    nome_da_thread,        // Um nome interno para a thread
//    tamanho_da_pilha,      // Memória reservada (geralmente 1024 bytes)
//    funcao_da_thread,      // O nome da função que tem o loop da thread
//    arg1, arg2, arg3,      // Argumentos (geralmente NULL)
//    prioridade,            // Número da prioridade (menor = mais
importante)
//    opcoes,                // 0 por padrão
//    delay_inicial          // 0 para iniciar na hora
//);

// declaratacao thread adc
K_THREAD_DEFINE(
    padeiro_tid,

```



```

// Tamanho de memória para cada thread
#define STACK_SIZE 1024

// prioridade da thread
#define PADEIRO_THREAD_PRIORITY 7
#define CLIENTE_THREAD_PRIORITY 5

// para alternar os modos do botao
static volatile int saldo_vitrine = 0;

// DECLARAÇÃO DOS SEMÁFOROS
#define CAPACIDADE_VITRINE 10
// (nome, valor_inicial, valor_maximo)
K_SEM_DEFINE(sem_paes, 0, CAPACIDADE_VITRINE);
K_SEM_DEFINE(sem_vagas, CAPACIDADE_VITRINE, CAPACIDADE_VITRINE);

// declaracao mutex
K_MUTEX_DEFINE(vitrine_mutex);

//////////////////////////////////// thread
padeiro////////////////////////////////////
void thread_padeiro_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        // se estiver cheio, task dorme para sempre
        k_sem_take(&sem_vagas, K_FOREVER);

        // mutex trava o uso da variavel
        k_mutex_lock(&vitrine_mutex, K_FOREVER);

        saldo_vitrine++;

        printk("[PADEIRO] Produziu 1 pao. Saldo atual da vitrine:
%d\n", saldo_vitrine);

        k_mutex_unlock(&vitrine_mutex);

        // adiciona um no semaforo
        k_sem_give(&sem_paes);

        k_msleep(1000); // Aguarda 1 segundo
    }
}

```



```

    }
}

//////////////////////////////// thread
cliente////////////////////////////////
void thread_cliente_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        // semaforo de se vitrine vazia, task dorme
        k_sem_take(&sem_paes, K_FOREVER);
        // mutex para uso da variavel
        k_mutex_lock(&vitrine_mutex, K_FOREVER);

        saldo_vitrine--;

        printk("[CLIENTE] Retirou 1 pao. Saldo atual da vitrine: %d\n",
saldo_vitrine);

        k_mutex_unlock(&vitrine_mutex);

        // libera 1 vaga para a vitrine
        k_sem_give(&sem_vagas);

        k_msleep(1500); // Aguarda 1.5 segundos
    }
}

//K_THREAD_DEFINE(
//    nome_da_thread,        // Um nome interno para a thread
//    tamanho_da_pilha,      // Memória reservada (geralmente 1024 bytes)
//    funcao_da_thread,      // O nome da função que tem o loop da thread
//    arg1, arg2, arg3,      // Argumentos (geralmente NULL)
//    prioridade,            // Número da prioridade (menor = mais
importante)
//    opcoes,                // 0 por padrão
//    delay_inicial          // 0 para iniciar na hora
//);

// declaratacao thread adc
K_THREAD_DEFINE(
    padeiro_tid,
    STACK_SIZE,

```

```

    thread_padeiro_fn,
    NULL,
    NULL,
    NULL,
    PADEIRO_THREAD_PRIORITY,
    0,
    0
);

// declaratacao thread accel
K_THREAD_DEFINE(
    cliente_tid,
    STACK_SIZE,
    thread_cliente_fn,
    NULL,
    NULL,
    NULL,
    CLIENTE_THREAD_PRIORITY,
    0,
    0
);

int main(void)
{
    printk("=====\n");
    printk(" PSI3441 - Atividade 7 - Parte 3 (Com Semaforos)\n");
    printk("=====\n\n");

    // O main entra em repouso e deixa as threads rodando
    while (1) {
        k_sleep(K_FOREVER);
    }
    return 0;
}

```

4. Referências e Links Externos

Utilize este campo para referenciar repositórios de controle de versão, vídeos demonstrativos ou documentos complementares.

- **Link do Repositório (GitHub/GitLab):**

<https://github.com/Djovanne/PSI3441---Arquitetura-de-Sistemas-Embarcados/tree/main/Atividade%207%20-%20Shared%20Resources>